NOVA

Deep Learning Model Training & HuggingFace Deployment Guide

PyTorch | TensorFlow Lite | Knowledge Distillation | INT8 Quantization | Version 1.0

Field	Value
Document Type	Deep Learning Model Training & Deployment Guide
Models Covered	Obstacle Detection, Currency Detection, Face Detection, Face Embedding
Training Framework	PyTorch 2.3+ with HuggingFace Transformers/Timm
Deployment Format	TensorFlow Lite INT8 (mobile) + PyTorch (HuggingFace Hub)
Compression Techniques	Knowledge Distillation (Hinton et al., 2015) + Post-Training Quantization (INT8)
HuggingFace Organisation	nova-assistive (to be created at huggingface.co/nova-assistive)
Target Hardware	Low-end Android (quad-core 1.4GHz, 2GB RAM, no NPU)
Reference Teacher Models	YOLOv8m (obstacle), EfficientNet-B4 (currency), ArcFace R100 (faces)
Reference Student Models	YOLOv8n / MobileNetV3-SSD (obstacle), MobileNetV3-S (currency), MobileFaceNet (faces)
Document Version	1.0 June 2026

1. Overview and Model Pipeline

1.1 Purpose of This Document

This document describes the complete pipeline for training, compressing, evaluating, and deploying the deep learning models that power NOVA's on-device computer vision features. It covers four model modules: obstacle detection (MOD-01), currency denomination classification (MOD-04), face detection (MOD-05 detect stage), and face embedding extraction (MOD-05 embed stage). Each model follows the same three-phase pipeline: train or fine-tune a student model using knowledge distillation from a stronger teacher, apply INT8 post-training quantization to produce a TFLite file for the mobile app, and publish both the full-precision PyTorch checkpoint and the quantized TFLite artifact to the HuggingFace Hub.

1.2 Why Knowledge Distillation + Quantization

The models must run on low-end Android devices with no NPU quad-core ARM Cortex-A53, 2 GB RAM, inference on CPU only. State-of-the-art teacher models (YOLOv8m, EfficientNet-B4, ArcFace R100) achieve excellent accuracy but are far too large and slow for this hardware. Two compression stages are applied in sequence:

- Knowledge Distillation (KD): A small student network is trained to reproduce the output distribution of the large teacher, not just the hard ground-truth labels. The soft probability distributions from the teacher carry richer information (inter-class relationships, confidence calibration) than one-hot labels alone. This transfers accuracy from the teacher into a network small enough to run on device.
- INT8 Post-Training Quantization (PTQ): After distillation, weights and activations are converted from 32-bit floating point to 8-bit integer. This reduces model size by approximately 4x and inference latency by 2-4x on ARM CPUs, with typical accuracy loss of less than 1-2% on most vision tasks.

The combined result is a model that is 8-16x smaller and 4-8x faster than the teacher, with accuracy competitive with models 2-3x its own size trained without distillation.

1.3 Repository Structure on HuggingFace Hub

All models are published under the HuggingFace organisation nova-assistive. Each model module has its own repository following a consistent naming convention:

HuggingFace Repo	Contents	Visibility
nova-assistive/obstacle-detection	PyTorch student checkpoint, TFLite INT8 model, COCO labels, model card	Public
nova-assistive/currency-detection	PyTorch student checkpoint, TFLite INT8 model, CFA franc labels, model card	Public
nova-assistive/face-detection	PyTorch BlazeFace student checkpoint, TFLite INT8 model, model card	Public
nova-assistive/face-embedding	PyTorch MobileFaceNet checkpoint, TFLite INT8 model, model card	Public
nova-assistive/nova-model-configs	Shared config YAMLs, training scripts, distillation utilities	Public

2. Environment Setup

2.1 Python Environment

```
# Create and activate a virtual environment
python -m venv nova_ml_env
source nova_ml_env/bin/activate  # Linux/Mac
# nova_ml_env\Scripts\activate   # Windows

# Install all training dependencies
pip install torch==2.3.0 torchvision==0.18.0 --index-url
https://download.pytorch.org/whl/cu121
pip install transformers==4.41.0
pip install timm==1.0.3
pip install ultralytics==8.2.0       # YOLOv8 training framework
pip install huggingface_hub==0.23.2
pip install datasets==2.19.0
pip install albumentations==1.4.7    # Data augmentation
pip install onnx==1.16.0
pip install onnxruntime==1.18.0
pip install tensorflow==2.16.1       # For TFLite conversion
pip install tf-nightly               # Latest TFLite converter features
pip install opencv-python==4.9.0.80
pip install Pillow==10.3.0
pip install numpy==1.26.4
pip install matplotlib==3.9.0
pip install scikit-learn==1.5.0
pip install tqdm==4.66.4
pip install wandb==0.17.0            # Experiment tracking
pip install pyyaml==6.0.1
```

2.2 HuggingFace CLI Setup

```
# Install HuggingFace CLI
pip install huggingface_hub[cli]

# Login with your HuggingFace token
huggingface-cli login
# Paste your HF token when prompted (Settings > Access Tokens on hf.co)

# Create the nova-assistive organisation on huggingface.co first,
# then create each repository:
huggingface-cli repo create nova-assistive/obstacle-detection --type model
huggingface-cli repo create nova-assistive/currency-detection --type model
huggingface-cli repo create nova-assistive/face-detection --type model
huggingface-cli repo create nova-assistive/face-embedding --type model
huggingface-cli repo create nova-assistive/nova-model-configs --type dataset
```

3. MOD-01: Obstacle Detection Model

3.1 Task Definition

Real-time multi-class object detection on camera frames captured from a chest-mounted Android device. The model must detect obstacles relevant to pedestrian navigation (people, vehicles, furniture, steps, poles) and output bounding boxes, class labels, and confidence scores at a minimum of 10 FPS on the reference low-end device.

3.2 Teacher Model

Property	Value
Architecture	YOLOv8m (medium)
Parameters	~25.9 million
Input size	640x640
Pre-trained weights	YOLOv8m COCO (ultralytics/assets)
mAP50 on COCO val	~50.2%
Role	Soft-label teacher for knowledge distillation only; not deployed to device

3.3 Student Model

Property	Value
Architecture	YOLOv8n (nano) smallest YOLOv8 variant
Parameters	~3.2 million
Input size	320x320 (reduced from 640 for speed)
Pre-trained weights	YOLOv8n COCO (ultralytics/assets) fine-tuned with distillation
Target mAP50 on COCO val	>= 30% (acceptable for assistive navigation; precision of alert matters more than recall)
Target inference latency	< 100ms per frame on reference device (TFLite INT8)
Target model size (TFLite INT8)	< 15 MB

3.4 Dataset

Training uses a combination of three datasets:

Dataset	Size	Purpose	Source
COCO 2017	118k train / 5k val images, 80 classes	Base object detection training	cocodataset.org
VisDrone 2019	6,471 train images, drone/pedestrian view	Dense pedestrian and small obstacle detection	github.com/VisDrone/VisDrone-Dataset
Custom Cameroonian street scenes	Target: 500+ images	Domain adaptation for local context (market stalls, motorbikes, open drains)	Collected by the team during field visits

The custom dataset is the critical differentiator for the Cameroonian deployment context. It should be annotated using LabelImg or Roboflow (free tier) with YOLO format bounding box labels. Minimum

500 images; aim for 100+ images per context-specific obstacle class (motorbike, market stall, open drain, low-hanging sign).

3.5 Knowledge Distillation Training Script

```
# scripts/train_obstacle_distillation.py
import torch
import torch.nn as nn
import torch.nn.functional as F
from ultralytics import YOLO
from torch.utils.data import DataLoader
import wandb

# ── Configuration ──────────────────────────────────────────
TEACHER_WEIGHTS = 'yolov8m.pt'
STUDENT_WEIGHTS = 'yolov8n.pt'
DATA_YAML       = 'configs/obstacle_data.yaml'
EPOCHS          = 100
IMG_SIZE        = 320
BATCH_SIZE      = 32
TEMPERATURE     = 4.0    # KD softening temperature (T in Hinton et al.)
ALPHA           = 0.7    # Weight on distillation loss
BETA            = 0.3    # Weight on ground-truth detection loss
LR              = 0.001
DEVICE          = 'cuda' if torch.cuda.is_available() else 'cpu'
PROJECT         = 'nova-obstacle-distillation'

# ── Loss Functions ─────────────────────────────────────────
class DistillationLoss(nn.Module):
    """
    Combined KD loss for object detection.
    For detection, we distil on the classification head logits.
    The localisation (box regression) head is trained purely on GT.
    """
    def __init__(self, temperature: float, alpha: float, beta: float):
        super().__init__()
        self.T = temperature
        self.alpha = alpha
        self.beta = beta

    def forward(
        self,
        student_cls_logits: torch.Tensor,
        teacher_cls_logits: torch.Tensor,
        student_box_loss: torch.Tensor,
        gt_cls_loss: torch.Tensor,
    ) -> torch.Tensor:
        # Soft targets from teacher (temperature-scaled)
        teacher_soft = F.softmax(teacher_cls_logits / self.T, dim=-1)
        student_log_soft = F.log_softmax(student_cls_logits / self.T, dim=-1)

        # KL divergence distillation loss (scaled by T^2 as per Hinton et al.)
        kd_loss = F.kl_div(
            student_log_soft,
            teacher_soft,
            reduction='batchmean',
        ) * (self.T ** 2)

        # Combined loss
        total = (
            self.alpha * kd_loss
            + self.beta * (gt_cls_loss + student_box_loss)
        )
        return total

# ── Training ───────────────────────────────────────────────
```

```
def train():
    wandb.init(project=PROJECT, config={
        'teacher': TEACHER_WEIGHTS, 'student': STUDENT_WEIGHTS,
        'temperature': TEMPERATURE, 'alpha': ALPHA, 'beta': BETA,
        'epochs': EPOCHS, 'img_size': IMG_SIZE,
    })

    # Load teacher (frozen  inference only)
    teacher = YOLO(TEACHER_WEIGHTS).to(DEVICE)
    teacher.model.eval()
    for param in teacher.model.parameters():
        param.requires_grad = False

    # Load student (trainable)
    student = YOLO(STUDENT_WEIGHTS)

    # YOLOv8 native training with custom callback to inject KD loss
    # See ultralytics docs for custom trainer override
    student.train(
        data=DATA_YAML,
        epochs=EPOCHS,
        imgsz=IMG_SIZE,
        batch=BATCH_SIZE,
        lr0=LR,
        device=DEVICE,
        project=PROJECT,
        name='student_kd_run',
        # Note: inject DistillationLoss via custom YOLOv8 Trainer subclass
        # See Section 3.6 for the custom Trainer implementation
    )

    wandb.finish()
    return student

if __name__ == '__main__':
    trained_student = train()
```

3.6 Custom YOLOv8 Trainer with KD Loss Injection

```
# scripts/kd_trainer.py
from ultralytics.engine.trainer import BaseTrainer
from ultralytics.models.yolo.detect import DetectionTrainer
from train_obstacle_distillation import DistillationLoss, TEMPERATURE, ALPHA,
BETA
import torch

class KDDetectionTrainer(DetectionTrainer):
    """
    Extends YOLOv8 DetectionTrainer to add knowledge distillation loss.
    The teacher model is loaded once and used only for forward passes.
    """
    def __init__(self, teacher_model, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.teacher = teacher_model
        self.kd_loss_fn = DistillationLoss(TEMPERATURE, ALPHA, BETA)

    def criterion(self, preds, batch):
        # Standard YOLOv8 detection loss on ground truth
        gt_loss, gt_loss_items = super().criterion(preds, batch)

        # Teacher forward pass (no grad)
        with torch.no_grad():
            teacher_preds = self.teacher.model(batch['img'])

        # Extract classification logits from prediction heads
        # preds structure: list of [batch, anchors, classes+5] tensors
```

```
        student_cls = preds[0][..., 5:]   # class logits
        teacher_cls = teacher_preds[0][..., 5:]

        # Align spatial dimensions if teacher/student differ
        if teacher_cls.shape != student_cls.shape:
            teacher_cls = torch.nn.functional.interpolate(
                teacher_cls.permute(0, 2, 1).unsqueeze(-1),
                size=(student_cls.shape[1], 1),
                mode='bilinear', align_corners=False,
            ).squeeze(-1).permute(0, 2, 1)

        kd_total = self.kd_loss_fn(
            student_cls, teacher_cls,
            gt_loss_items[1],  # box regression loss component
            gt_loss_items[0],  # classification loss component
        )

        return kd_total, gt_loss_items
```

3.7 Data Configuration YAML

```
# configs/obstacle_data.yaml
path: ./datasets/obstacle_combined
train: images/train
val: images/val
test: images/test

nc: 83  # 80 COCO + 3 custom Cameroonian classes
names:
  # COCO classes 0-79 (standard)
  0: person
  1: bicycle
  2: car
  # ... (full COCO list)
  79: toothbrush
  # Custom Cameroonian context classes
  80: open_drain
  81: market_stall
  82: low_hanging_sign

# Augmentation config (applied during training)
augment: true
hsv_h: 0.015
hsv_s: 0.7
hsv_v: 0.4
degrees: 5.0       # small rotation only (chest-mounted camera)
translate: 0.1
scale: 0.5
flipud: 0.0        # vertical flip disabled (chest-mount, gravity is constant)
fliplr: 0.5
mosaic: 1.0        # mosaic augmentation for small object training
mixup: 0.1
```

4. MOD-04: Currency Detection Model

4.1 Task Definition

Single-label image classification of Central African CFA franc banknotes into five denomination classes: 500, 1000, 2000, 5000, and 10000 FCFA. The model must handle both obverse and reverse faces of each note, varying lighting conditions, and partial occlusion (e.g., a note held by fingers). Inference is triggered on a still frame captured on user command, so real-time throughput is not required latency budget is 500ms.

4.2 Teacher and Student Models

Property	Teacher	Student
Architecture	EfficientNet-B4	MobileNetV3-Small
Parameters	~19 million	~2.5 million
Input size	380x380	224x224
Pre-trained weights	ImageNet-21k (HuggingFace timm)	ImageNet-1k (HuggingFace timm)
Output classes	5 (CFA denominations)	5 (CFA denominations)
Target accuracy (val)	>= 97%	>= 93% (post-distillation)
Target TFLite INT8 size	N/A (not deployed)	< 10 MB

4.3 Dataset Collection Strategy

No public dataset exists for CFA franc banknotes. The team must construct one. Target: minimum 300 images per class per side = 3,000 images total before augmentation.

Collection Method	Description	Target Count
Direct photography	Photograph each note denomination (front and back) under different lighting: natural daylight, indoor fluorescent, torch, low light. Use multiple phones to capture camera variation.	100 images per denomination per side
BEAC official scans	Download official high-resolution note scans from the Bank of Central African States (BEAC) website for use as base images for synthetic augmentation.	10 images per denomination per side
Synthetic augmentation	Apply heavy augmentation (random rotation, perspective warp, brightness, motion blur, partial occlusion masks) to BEAC scans to generate additional training variety.	200 synthetic images per denomination per side
Real-world context	Photograph notes as they appear in actual use: in a hand, on a table, partially folded, crumpled, with wear marks.	50 images per denomination per side

4.4 Distillation Training Script

```
# scripts/train_currency_distillation.py
import torch
import torch.nn as nn
import torch.nn.functional as F
import timm
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
```

```
import wandb
from pathlib import Path

# ── Config ──────────────────────────────────────────────────
TEACHER_MODEL   = 'efficientnet_b4'
STUDENT_MODEL   = 'mobilenetv3_small_100'
NUM_CLASSES     = 5
TEMPERATURE     = 5.0
ALPHA           = 0.8   # Higher alpha = more weight on teacher's knowledge
EPOCHS          = 80
BATCH_SIZE      = 32
LR              = 3e-4
DATA_DIR        = Path('./datasets/cfa_currency')
DEVICE          = 'cuda' if torch.cuda.is_available() else 'cpu'

# ── Transforms ──────────────────────────────────────────────
train_transforms = transforms.Compose([
    transforms.Resize((256, 256)),
    transforms.RandomCrop(224),
    transforms.RandomHorizontalFlip(),
    transforms.RandomVerticalFlip(p=0.2),
    transforms.ColorJitter(brightness=0.4, contrast=0.4, saturation=0.3),
    transforms.RandomRotation(degrees=30),
    transforms.RandomPerspective(distortion_scale=0.3, p=0.5),
    transforms.GaussianBlur(kernel_size=3, sigma=(0.1, 1.5)),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225]),
])

val_transforms = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225]),
])

# ── Dataset ─────────────────────────────────────────────────
# Expected folder structure:
# datasets/cfa_currency/
#   train/fcfa_500/, train/fcfa_1000/, ... train/fcfa_10000/
#   val/fcfa_500/, val/fcfa_1000/, ...
#   test/fcfa_500/, ...

train_dataset = datasets.ImageFolder(DATA_DIR / 'train',
transform=train_transforms)
val_dataset   = datasets.ImageFolder(DATA_DIR / 'val',
transform=val_transforms)
train_loader  = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True,
num_workers=4)
val_loader    = DataLoader(val_dataset,   batch_size=BATCH_SIZE, shuffle=False,
num_workers=4)

# ── Models ──────────────────────────────────────────────────
# Teacher: pre-trained on ImageNet, fine-tuned first on CFA data
teacher = timm.create_model(TEACHER_MODEL, pretrained=True,
num_classes=NUM_CLASSES)
teacher = teacher.to(DEVICE)

# Student: smaller model, trained with KD
student = timm.create_model(STUDENT_MODEL, pretrained=True,
num_classes=NUM_CLASSES)
student = student.to(DEVICE)

# ── Loss ────────────────────────────────────────────────────
ce_loss = nn.CrossEntropyLoss()  # Ground-truth loss
kl_loss = nn.KLDivLoss(reduction='batchmean')  # Distillation loss
```

```
def distillation_loss(student_logits, teacher_logits, labels):
    # Soft targets
    soft_teacher = F.softmax(teacher_logits / TEMPERATURE, dim=1)
    soft_student = F.log_softmax(student_logits / TEMPERATURE, dim=1)
    kd = kl_loss(soft_student, soft_teacher) * (TEMPERATURE ** 2)
    # Hard targets
    hard = ce_loss(student_logits, labels)
    return ALPHA * kd + (1 - ALPHA) * hard

# ── Phase 1: Fine-tune teacher on CFA data ──────────────────
def finetune_teacher():
    print('Phase 1: Fine-tuning teacher on CFA currency data...')
    optimizer = torch.optim.AdamW(teacher.parameters(), lr=1e-4,
weight_decay=1e-4)
    scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=30)
    for epoch in range(30):
        teacher.train()
        for imgs, labels in train_loader:
            imgs, labels = imgs.to(DEVICE), labels.to(DEVICE)
            optimizer.zero_grad()
            loss = ce_loss(teacher(imgs), labels)
            loss.backward()
            optimizer.step()
        scheduler.step()
    teacher.eval()
    for p in teacher.parameters(): p.requires_grad = False
    print('Teacher fine-tuning complete.')

# ── Phase 2: Distill teacher into student ──────────────────
def distill_student():
    print('Phase 2: Knowledge distillation into student...')
    optimizer = torch.optim.AdamW(student.parameters(), lr=LR,
weight_decay=1e-4)
    scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer,
T_max=EPOCHS)
    wandb.init(project='nova-currency-distillation')
    best_val_acc = 0.0

    for epoch in range(EPOCHS):
        student.train()
        total_loss = 0.0
        for imgs, labels in train_loader:
            imgs, labels = imgs.to(DEVICE), labels.to(DEVICE)
            with torch.no_grad():
                teacher_logits = teacher(imgs)
            student_logits = student(imgs)
            loss = distillation_loss(student_logits, teacher_logits, labels)
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()
            total_loss += loss.item()
        scheduler.step()

        # Validation
        val_acc = evaluate(student, val_loader)
        wandb.log({'epoch': epoch, 'val_acc': val_acc, 'loss': total_loss})
        print(f'Epoch {epoch+1}/{EPOCHS} | Loss: {total_loss:.4f} | Val Acc:
{val_acc:.4f}')

        if val_acc > best_val_acc:
            best_val_acc = val_acc
            torch.save(student.state_dict(),
'checkpoints/currency_student_best.pth')
            print(f'  -> New best model saved (val_acc={val_acc:.4f})')

    wandb.finish()
```

```
def evaluate(model, loader):
    model.eval()
    correct, total = 0, 0
    with torch.no_grad():
        for imgs, labels in loader:
            imgs, labels = imgs.to(DEVICE), labels.to(DEVICE)
            preds = model(imgs).argmax(dim=1)
            correct += (preds == labels).sum().item()
            total += labels.size(0)
    return correct / total

if __name__ == '__main__':
    finetune_teacher()
    distill_student()
```

5. MOD-05: Face Detection and Embedding Models

5.1 Overview

Face recognition in NOVA is a two-stage pipeline. Stage 1 (face detection) locates and crops faces in the camera frame using a lightweight on-device detector. Stage 2 (face embedding) converts each detected face crop into a 128-dimensional embedding vector, which is compared against the enrolled gallery using cosine similarity. Both stages are deployed as TFLite models on the device for small galleries; the embedding comparison is offloaded to the FastAPI backend for larger galleries (>20 contacts).

5.2 Stage 1: Face Detection BlazeFace Student

Property	Value
Teacher model	RetinaFace (ResNet-50 backbone) high-accuracy face detector
Student model	BlazeFace (MediaPipe) ultra-lightweight face detector
Student parameters	~220,000 (extremely lightweight)
Input size	128x128
Output	Bounding boxes + confidence scores for detected faces
Training data	WIDER FACE dataset (32,203 images, 393,703 face annotations)
Target precision	>= 90% on WIDER FACE easy/medium sets
Target TFLite INT8 size	< 5 MB

```
# scripts/train_face_detection.py
# BlazeFace is available as a pre-trained model from MediaPipe.
# For NOVA, we fine-tune it on WIDER FACE for robustness.
# Full distillation from RetinaFace is optional if BlazeFace pre-trained
# accuracy is sufficient on validation set.

import torch
from torch.utils.data import DataLoader
from datasets import load_dataset  # HuggingFace datasets

# Load WIDER FACE from HuggingFace Hub
wider_face = load_dataset('wider_face', split='train')

# Fine-tune BlazeFace on WIDER FACE
# See MediaPipe BlazeFace PyTorch implementation:
# github.com/hollance/BlazeFace-PyTorch

# After fine-tuning, export to ONNX then convert to TFLite:
# See Section 7 for the full conversion pipeline.
```

5.3 Stage 2: Face Embedding MobileFaceNet

Property	Value
Teacher model	ArcFace with ResNet-100 backbone state-of-the-art face recognition
Student model	MobileFaceNet lightweight ArcFace-trained embedding model
Student parameters	~1.0 million
Input size	112x112 (aligned face crop)

Property	Value
Output	512-dimensional L2-normalised embedding vector
Training data	MS1MV2 (5.8M images, 85k identities) standard ArcFace training set
Distance metric	Cosine similarity (dot product of L2-normalised vectors)
Match threshold	0.75 cosine similarity (configurable in FaceService)
Target LFW accuracy	>= 99.0% (standard face recognition benchmark)
Target TFLite INT8 size	< 12 MB

```
# scripts/train_face_embedding.py
# MobileFaceNet with ArcFace loss  distilled from ArcFace ResNet-100
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader
import wandb

# ── ArcFace Loss (margin-based softmax) ──────────────────────
class ArcFaceLoss(nn.Module):
    def __init__(self, num_classes: int, embedding_dim: int,
                 scale: float = 64.0, margin: float = 0.5):
        super().__init__()
        self.scale = scale
        self.margin = margin
        self.weight = nn.Parameter(
            torch.FloatTensor(num_classes, embedding_dim))
        nn.init.xavier_uniform_(self.weight)

    def forward(self, embeddings: torch.Tensor, labels: torch.Tensor):
        # L2 normalise embeddings and weights
        emb_norm = F.normalize(embeddings, p=2, dim=1)
        w_norm   = F.normalize(self.weight,    p=2, dim=1)
        cosine = F.linear(emb_norm, w_norm)  # [N, C]

        # Add angular margin to target class
        theta = torch.acos(cosine.clamp(-1 + 1e-6, 1 - 1e-6))
        target_logits = torch.cos(theta + self.margin)
        one_hot = torch.zeros_like(cosine)
        one_hot.scatter_(1, labels.view(-1, 1), 1.0)
        output = cosine * (1 - one_hot) + target_logits * one_hot
        output *= self.scale
        return F.cross_entropy(output, labels)

# ── KD Loss for Embeddings ──────────────────────────────────
class EmbeddingDistillationLoss(nn.Module):
    """
    Distil embedding space: minimise MSE between student and teacher
    embedding vectors. Encourages the student to produce embeddings
    in the same metric space as the teacher.
    """
    def __init__(self, alpha: float = 0.6):
        super().__init__()
        self.alpha = alpha
        self.mse = nn.MSELoss()

    def forward(
        self,
        student_emb: torch.Tensor,
        teacher_emb: torch.Tensor,
        arcface_loss: torch.Tensor,
    ) -> torch.Tensor:
```

```
        # Normalise both embedding spaces to unit sphere
        s = F.normalize(student_emb, p=2, dim=1)
        t = F.normalize(teacher_emb, p=2, dim=1)
        emb_loss = self.mse(s, t)
        return self.alpha * emb_loss + (1 - self.alpha) * arcface_loss

# ── MobileFaceNet Architecture ──────────────────────────────
# Use the reference implementation from:
# github.com/cavalleria/cavaface.pytorch (MobileFaceNet)
# or load directly from HuggingFace:
# from huggingface_hub import hf_hub_download
# weights = hf_hub_download('minchul/cvlface_adaface_mobilenetv2_casia',
'model.ckpt')

# ── Training Loop (simplified) ──────────────────────────────
def train_embedding(teacher, student, loader, epochs=50):
    arcface = ArcFaceLoss(num_classes=85000, embedding_dim=512).to(DEVICE)
    kd_loss = EmbeddingDistillationLoss(alpha=0.6)
    optimizer = torch.optim.SGD(
        list(student.parameters()) + list(arcface.parameters()),
        lr=0.1, momentum=0.9, weight_decay=5e-4,
    )
    scheduler = torch.optim.lr_scheduler.MultiStepLR(
        optimizer, milestones=[20, 35, 45], gamma=0.1
    )
    wandb.init(project='nova-face-embedding')
    for epoch in range(epochs):
        student.train()
        for imgs, labels in loader:
            imgs, labels = imgs.to(DEVICE), labels.to(DEVICE)
            with torch.no_grad():
                teacher_emb = teacher(imgs)  # Teacher embeddings
            student_emb = student(imgs)
            af_loss = arcface(student_emb, labels)
            loss = kd_loss(student_emb, teacher_emb, af_loss)
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()
        scheduler.step()
        wandb.log({'epoch': epoch, 'loss': loss.item()})
```

6. Model Evaluation

6.1 Evaluation Metrics per Model

Model	Primary Metric	Secondary Metrics	Acceptance Threshold
Obstacle Detection (student)	mAP50 on COCO val + custom Cameroonian test set	FPS on reference device, model size, precision/recall per class	mAP50 >= 30%; FPS >= 10; size < 15MB
Currency Detection (student)	Top-1 accuracy on held-out test set	Per-class accuracy (each denomination), confusion matrix, accuracy at confidence >= 0.85	Overall accuracy >= 93%; per-class >= 90%
Face Detection (BlazeFace)	Precision and Recall on WIDER FACE easy/medium	FPS on reference device, false positive rate	Precision >= 90%, Recall >= 85%
Face Embedding (MobileFaceNet)	LFW verification accuracy, TAR@FAR=1e-3	Cosine similarity distribution (genuine vs impostor pairs)	LFW accuracy >= 99.0%; TAR@FAR=1e-3 >= 90%

6.2 Evaluation Script

```
# scripts/evaluate_models.py
import torch
import numpy as np
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix
from sklearn.metrics import roc_curve

def evaluate_currency_model(model, test_loader, class_names, device):
    model.eval()
    all_preds, all_labels, all_confs = [], [], []

    with torch.no_grad():
        for imgs, labels in test_loader:
            imgs = imgs.to(device)
            logits = model(imgs)
            probs = torch.softmax(logits, dim=1)
            confs, preds = probs.max(dim=1)
            all_preds.extend(preds.cpu().numpy())
            all_labels.extend(labels.numpy())
            all_confs.extend(confs.cpu().numpy())

    print('=== Currency Detection Evaluation ===')
    print(classification_report(all_labels, all_preds,
target_names=class_names))
    print('Confusion Matrix:')
    print(confusion_matrix(all_labels, all_preds))

    # Accuracy at confidence >= 0.85 (matches NFR-04-03 threshold)
    high_conf_mask = np.array(all_confs) >= 0.85
    high_conf_acc = accuracy_score(
        np.array(all_labels)[high_conf_mask],
        np.array(all_preds)[high_conf_mask]
    )
    coverage = high_conf_mask.mean()
    print(f'Accuracy @ conf>=0.85: {high_conf_acc:.4f} (coverage:
{coverage:.2%})')

def evaluate_face_verification(model, pair_loader, threshold, device):
    """LFW-style verification: given pairs, predict same/different."""
```

```
    model.eval()
    similarities, is_same = [], []

    with torch.no_grad():
        for img1, img2, same in pair_loader:
            emb1 = torch.nn.functional.normalize(model(img1.to(device)), p=2,
dim=1)
            emb2 = torch.nn.functional.normalize(model(img2.to(device)), p=2,
dim=1)
            sim = (emb1 * emb2).sum(dim=1).cpu().numpy()
            similarities.extend(sim)
            is_same.extend(same.numpy())

    similarities = np.array(similarities)
    is_same = np.array(is_same)
    preds = similarities >= threshold
    acc = accuracy_score(is_same, preds)

    # TAR @ FAR = 1e-3
    fpr, tpr, _ = roc_curve(is_same, similarities)
    tar_at_far = tpr[np.searchsorted(fpr, 1e-3)]

    print(f'LFW Accuracy @ threshold {threshold}: {acc:.4f}')
    print(f'TAR @ FAR=1e-3: {tar_at_far:.4f}')
```

7. TFLite Conversion Pipeline

7.1 PyTorch → ONNX → TFLite INT8

All models follow the same three-step conversion pipeline: PyTorch checkpoint → ONNX → TFLite INT8. This two-step process is necessary because TensorFlow's converter does not natively import PyTorch models ONNX acts as the interchange format.

```
# scripts/convert_to_tflite.py
import torch
import torch.onnx
import onnx
import numpy as np
import tensorflow as tf
from pathlib import Path

def export_pytorch_to_onnx(
    model: torch.nn.Module,
    input_shape: tuple,
    output_path: str,
    opset: int = 17,
):
    """Step 1: Export PyTorch model to ONNX format."""
    model.eval()
    dummy_input = torch.randn(*input_shape)
    torch.onnx.export(
        model,
        dummy_input,
        output_path,
        opset_version=opset,
        input_names=['input'],
        output_names=['output'],
        dynamic_axes={'input': {0: 'batch_size'}, 'output': {0: 'batch_size'}},
        do_constant_folding=True,
    )
    # Validate ONNX model
    onnx_model = onnx.load(output_path)
    onnx.checker.check_model(onnx_model)
    print(f'ONNX model exported and validated: {output_path}')
    return output_path


def convert_onnx_to_tflite_int8(
    onnx_path: str,
    tflite_output_path: str,
    representative_dataset_gen,
):
    """
    Step 2: Convert ONNX to TFLite with full INT8 post-training quantization.
    representative_dataset_gen is a generator that yields sample input batches
    from the real training/validation data. This is required for PTQ
calibration.
    """
    # Step 2a: ONNX → TF SavedModel using onnx-tf
    import onnx_tf
    from onnx_tf.backend import prepare

    onnx_model = onnx.load(onnx_path)
    tf_rep = prepare(onnx_model)
    saved_model_dir = onnx_path.replace('.onnx', '_saved_model')
    tf_rep.export_graph(saved_model_dir)
    print(f'TF SavedModel exported: {saved_model_dir}')

    # Step 2b: TF SavedModel → TFLite INT8
    converter = tf.lite.TFLiteConverter.from_saved_model(saved_model_dir)
```

```
    # Enable full INT8 quantization
    converter.optimizations = [tf.lite.Optimize.DEFAULT]
    converter.representative_dataset = representative_dataset_gen
    converter.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
    converter.inference_input_type  = tf.int8   # Input tensor type
    converter.inference_output_type = tf.int8   # Output tensor type

    tflite_model = converter.convert()
    Path(tflite_output_path).write_bytes(tflite_model)
    size_mb = len(tflite_model) / (1024 * 1024)
    print(f'TFLite INT8 model saved: {tflite_output_path} ({size_mb:.2f} MB)')
    return tflite_output_path


def make_representative_dataset(val_loader, n_samples: int = 200):
    """
    Creates a calibration dataset generator for PTQ.
    PTQ requires ~100-200 representative input samples to calibrate
    the quantization scale factors for each layer.
    """
    samples = []
    for imgs, _ in val_loader:
        samples.append(imgs.numpy())
        if len(samples) * imgs.shape[0] >= n_samples:
            break

    def generator():
        for batch in samples:
            for img in batch:
                yield [img[np.newaxis, ...].astype(np.float32)]

    return generator


# ── Run conversion for all NOVA models ──────────────────────
if __name__ == '__main__':
    models_to_convert = [
        {
            'name': 'obstacle_detection',
            'checkpoint': 'checkpoints/obstacle_student_best.pth',
            'input_shape': (1, 3, 320, 320),
            'onnx_path': 'exports/obstacle_detection.onnx',
            'tflite_path': 'exports/obstacle_detection_v1.tflite',
        },
        {
            'name': 'currency_detection',
            'checkpoint': 'checkpoints/currency_student_best.pth',
            'input_shape': (1, 3, 224, 224),
            'onnx_path': 'exports/currency_detection.onnx',
            'tflite_path': 'exports/currency_detection_v1.tflite',
        },
        {
            'name': 'face_embedding',
            'checkpoint': 'checkpoints/face_embedding_best.pth',
            'input_shape': (1, 3, 112, 112),
            'onnx_path': 'exports/face_embedding.onnx',
            'tflite_path': 'exports/face_embedding_v1.tflite',
        },
    ]

    for cfg in models_to_convert:
        print(f'\nConverting {cfg["name"]}...')
        export_pytorch_to_onnx(
            model=torch.load(cfg['checkpoint']),
            input_shape=cfg['input_shape'],
            output_path=cfg['onnx_path'],
```

```
        )
        # Pass your val_loader here for calibration
        # convert_onnx_to_tflite_int8(cfg['onnx_path'], cfg['tflite_path'], gen)
```

7.2 Verify TFLite Model on Device Benchmark

```
# scripts/benchmark_tflite.py
# Run this on an Android device via ADB or on a Linux machine
# to verify latency before pushing to HuggingFace.
import numpy as np
import time
import tensorflow as tf

def benchmark_tflite(tflite_path: str, input_shape: tuple, n_runs: int = 100):
    interpreter = tf.lite.Interpreter(model_path=tflite_path)
    interpreter.allocate_tensors()

    input_details  = interpreter.get_input_details()
    output_details = interpreter.get_output_details()

    # Warm-up
    dummy = np.random.randint(-128, 127, input_shape, dtype=np.int8)
    interpreter.set_tensor(input_details[0]['index'], dummy)
    for _ in range(5):
        interpreter.invoke()

    # Benchmark
    latencies = []
    for _ in range(n_runs):
        interpreter.set_tensor(input_details[0]['index'], dummy)
        start = time.perf_counter()
        interpreter.invoke()
        latencies.append((time.perf_counter() - start) * 1000)  # ms

    p50 = np.percentile(latencies, 50)
    p95 = np.percentile(latencies, 95)
    print(f'{tflite_path}')
    print(f'  Median latency: {p50:.1f}ms | P95: {p95:.1f}ms')
    print(f'  Equivalent FPS (median): {1000/p50:.1f}')

if __name__ == '__main__':
    benchmark_tflite('exports/obstacle_detection_v1.tflite', (1, 320, 320, 3))
    benchmark_tflite('exports/currency_detection_v1.tflite', (1, 224, 224, 3))
    benchmark_tflite('exports/face_embedding_v1.tflite',     (1, 112, 112, 3))
```

8. HuggingFace Hub Publishing Pipeline

8.1 Repository File Structure

Each HuggingFace model repository follows this standard structure:

```
nova-assistive/obstacle-detection/
├── README.md                        # Model card (auto-generated, see Section
8.3)
├── config.json                      # Model metadata (architecture, classes,
thresholds)
├── pytorch/
│   └── obstacle_student_best.pth    # Full-precision PyTorch checkpoint
├── tflite/
│   └── obstacle_detection_v1.tflite # INT8 quantized TFLite model
├── labels/
│   └── coco_labels_extended.txt     # 83-class label file
└── evaluation/
    └── results.json                 # mAP, FPS, size benchmark results
```

8.2 Publishing Script

```
# scripts/push_to_huggingface.py
from huggingface_hub import HfApi, create_repo
from pathlib import Path
import json, hashlib, datetime

api = HfApi()
ORG = 'nova-assistive'

def compute_sha256(file_path: str) -> str:
    sha256 = hashlib.sha256()
    with open(file_path, 'rb') as f:
        for chunk in iter(lambda: f.read(8192), b''):
            sha256.update(chunk)
    return sha256.hexdigest()

def publish_model(
    repo_name: str,
    pytorch_checkpoint: str,
    tflite_model: str,
    labels_file: str,
    config: dict,
    evaluation_results: dict,
    version: str,
):
    repo_id = f'{ORG}/{repo_name}'

    # Create repo if it doesn't exist
    try:
        create_repo(repo_id, repo_type='model', exist_ok=True)
    except Exception as e:
        print(f'Repo already exists or error: {e}')

    # Compute checksum of TFLite file (used by backend model registry)
    tflite_checksum = compute_sha256(tflite_model)
    config['tflite_checksum'] = tflite_checksum
    config['version'] = version
    config['published_at'] = datetime.datetime.utcnow().isoformat()

    # Upload PyTorch checkpoint
    api.upload_file(
        path_or_fileobj=pytorch_checkpoint,
        path_in_repo=f'pytorch/{Path(pytorch_checkpoint).name}',
        repo_id=repo_id,
        commit_message=f'Upload PyTorch checkpoint v{version}',
```

```
    )

    # Upload TFLite model
    api.upload_file(
        path_or_fileobj=tflite_model,
        path_in_repo=f'tflite/{Path(tflite_model).name}',
        repo_id=repo_id,
        commit_message=f'Upload TFLite INT8 model v{version}',
    )

    # Upload labels
    api.upload_file(
        path_or_fileobj=labels_file,
        path_in_repo=f'labels/{Path(labels_file).name}',
        repo_id=repo_id,
        commit_message='Upload label file',
    )

    # Upload config.json
    config_json = json.dumps(config, indent=2).encode()
    api.upload_file(
        path_or_fileobj=config_json,
        path_in_repo='config.json',
        repo_id=repo_id,
        commit_message=f'Update config for v{version}',
    )

    # Upload evaluation results
    results_json = json.dumps(evaluation_results, indent=2).encode()
    api.upload_file(
        path_or_fileobj=results_json,
        path_in_repo='evaluation/results.json',
        repo_id=repo_id,
        commit_message=f'Upload evaluation results for v{version}',
    )

    print(f'Model published: https://huggingface.co/{repo_id}')
    return tflite_checksum


# ── Publish all NOVA models ──────────────────────────────
if __name__ == '__main__':
    # Obstacle Detection
    checksum = publish_model(
        repo_name='obstacle-detection',
        pytorch_checkpoint='checkpoints/obstacle_student_best.pth',
        tflite_model='exports/obstacle_detection_v1.tflite',
        labels_file='labels/coco_labels_extended.txt',
        config={
            'module_id': 'MOD-01',
            'architecture': 'YOLOv8n',
            'input_size': [320, 320],
            'num_classes': 83,
            'distilled_from': 'YOLOv8m',
            'near_threshold_m': 1.5,
            'warning_threshold_m': 3.0,
            'confidence_threshold': 0.55,
            'suppression_seconds': 2.0,
        },
        evaluation_results={},  # Fill from evaluate_models.py output
        version='1.0.0',
    )
    print(f'Obstacle TFLite checksum: {checksum}')
    # Register this checksum in the backend model_registry table

    # Currency Detection
```

```
    publish_model(
        repo_name='currency-detection',
        pytorch_checkpoint='checkpoints/currency_student_best.pth',
        tflite_model='exports/currency_detection_v1.tflite',
        labels_file='labels/cfa_labels.txt',
        config={
            'module_id': 'MOD-04',
            'architecture': 'MobileNetV3-Small',
            'input_size': [224, 224],
            'num_classes': 5,
            'class_names':
['fcfa_500','fcfa_1000','fcfa_2000','fcfa_5000','fcfa_10000'],
            'spoken_labels': {
                'fcfa_500':   'Five hundred francs CFA',
                'fcfa_1000':  'One thousand francs CFA',
                'fcfa_2000':  'Two thousand francs CFA',
                'fcfa_5000':  'Five thousand francs CFA',
                'fcfa_10000': 'Ten thousand francs CFA',
            },
            'distilled_from': 'EfficientNet-B4',
            'confidence_threshold': 0.85,
        },
        evaluation_results={},
        version='1.0.0',
    )

    # Face Embedding
    publish_model(
        repo_name='face-embedding',
        pytorch_checkpoint='checkpoints/face_embedding_best.pth',
        tflite_model='exports/face_embedding_v1.tflite',
        labels_file='labels/empty.txt',  # No class labels for embedding models
        config={
            'module_id': 'MOD-05-embed',
            'architecture': 'MobileFaceNet',
            'input_size': [112, 112],
            'embedding_dim': 512,
            'distilled_from': 'ArcFace-R100',
            'match_threshold': 0.75,
            'distance_metric': 'cosine',
        },
        evaluation_results={},
        version='1.0.0',
    )
```

9. Model Card Template

Each HuggingFace repository requires a README.md model card. Below is the template used for all NOVA models. Replace placeholders in brackets for each specific model.

```
---
language:
  - en
  - fr
license: apache-2.0
tags:
  - assistive-technology
  - computer-vision
  - object-detection
  - tflite
  - knowledge-distillation
  - quantization
  - android
  - cameroon
  - nova-assistive
datasets:
  - [dataset name e.g. coco, wider_face]
metrics:
  - [primary metric e.g. map50, accuracy]
---

# NOVA  [Model Name] ([Module ID])

**Part of the NOVA (Navigational Object and Voice Assistant) project.**
Low-cost assistive vision system for blind and visually impaired users in
Cameroon.
University of Buea, Department of Computer Engineering, 2025/2026.

## Model Description

[One paragraph describing what this model does, its architecture,
and how it was trained. Include teacher and student architecture names.]

## Training Details

| Property | Value |
|---|---|
| Teacher model | [e.g. YOLOv8m] |
| Student model | [e.g. YOLOv8n] |
| Training technique | Knowledge Distillation (Hinton et al., 2015) + INT8 PTQ |
| Training data | [Dataset names and sizes] |
| Training epochs | [N] |
| KD Temperature | [T] |
| KD Alpha | [a] |
| Input size | [WxH] |

## Evaluation Results

| Metric | Value |
|---|---|
| [Primary metric] | [Value] |
| TFLite INT8 model size | [X] MB |
| Inference latency (reference device, CPU) | [X] ms |
| Equivalent FPS | [X] FPS |

## Files

| File | Description |
|---|---|
| `pytorch/[checkpoint].pth` | Full-precision PyTorch student checkpoint |
```

````
| `tflite/[model].tflite` | INT8 quantized TFLite model for Android deployment |
| `labels/[labels].txt` | Class label file |
| `config.json` | Model configuration and deployment parameters |
| `evaluation/results.json` | Full evaluation results |

## Usage in NOVA (Flutter/TFLite)

```dart
// Load via tflite_flutter
final interpreter = await Interpreter.fromAsset(
  'assets/models/[model_filename].tflite',
);
```

## Citation

If you use this model, please cite:
```
[Team names] (2026). NOVA: Navigational Object and Voice Assistant.
University of Buea, Faculty of Engineering and Technology.
Department of Computer Engineering.
```

## Limitations

- Trained primarily on COCO and [additional datasets]; performance may degrade
  on object categories not well-represented in training data.
- Optimised for chest-height camera perspective (approximately 1.2–1.5m above
ground).
- INT8 quantization may cause slight accuracy degradation (~1–2%) vs FP32
baseline.
- Currency detection is specific to BEAC CFA franc series; other currency zones
  are not supported.
````

10. Connecting HuggingFace to the Backend Model Registry

After publishing a new model version to HuggingFace, the TFLite file checksum and metadata must be registered in the backend PostgreSQL model_registry table so the mobile app can discover and download it. This is a manual step run by a developer after each model release.

```
# scripts/register_model_in_backend.py
# Run this after push_to_huggingface.py succeeds.
import requests
import json

BACKEND_URL = 'https://api.nova-assistive.cm'
ADMIN_TOKEN = 'your_admin_jwt_token'  # Never commit this  use env var

def register_model(
    module_id: str,
    version: str,
    filename: str,
    checksum: str,
    notes: str,
):
    """
    Insert a new row into model_registry and set it as active.
    The mobile app will discover this on next launch and download the model.
    """
    response = requests.post(
        f'{BACKEND_URL}/models/register',
        headers={'Authorization': f'Bearer {ADMIN_TOKEN}'},
        json={
            'module_id': module_id,
            'version': version,
            'filename': filename,
            'checksum': checksum,
            'is_active': True,
            'notes': notes,
        }
    )
    if response.status_code == 201:
        print(f'Model registered: {module_id} v{version}')
    else:
        print(f'Registration failed: {response.status_code} {response.text}')


if __name__ == '__main__':
    register_model(
        module_id='MOD-01',
        version='1.0.0',
        filename='obstacle_detection_v1.tflite',
        checksum='<sha256 from push_to_huggingface.py output>',
        notes='Initial release. YOLOv8n distilled from YOLOv8m. 83 classes.',
    )
    register_model(
        module_id='MOD-04',
        version='1.0.0',
        filename='currency_detection_v1.tflite',
        checksum='<sha256>',
        notes='Initial release. MobileNetV3-S distilled from EfficientNet-B4.',
    )
    register_model(
        module_id='MOD-05-embed',
        version='1.0.0',
        filename='face_embedding_v1.tflite',
        checksum='<sha256>',
        notes='Initial release. MobileFaceNet distilled from ArcFace R100.',
    )
```

11. Full Pipeline Summary

Step	Script / Tool	Input	Output	Where It Goes
1. Collect data	LabelImg / Roboflow	Raw images	Annotated dataset in YOLO/ImageFolder format	Local datasets/ folder
2. Fine-tune teacher	train_currency_distillation.py (Phase 1)	ImageNet pre-trained teacher, CFA images	Fine-tuned teacher checkpoint	checkpoints/
3. Distil student	train_obstacle_distillation.py / train_currency_distillation.py (Phase 2)	Teacher checkpoint + training data	Student checkpoint (best val)	checkpoints/
4. Evaluate student	evaluate_models.py	Student checkpoint + test data	Accuracy / mAP / LFW metrics	evaluation/results.json
5. Export to ONNX	convert_to_tflite.py (Step 1)	Student PyTorch checkpoint	ONNX model file	exports/
6. Convert to TFLite INT8	convert_to_tflite.py (Step 2)	ONNX model + calibration data	Quantized .tflite model	exports/
7. Benchmark TFLite	benchmark_tflite.py	TFLite model	Latency / FPS measurements	Console / results.json
8. Publish to HuggingFace	push_to_huggingface.py	Checkpoint + TFLite + labels + config	Published HF model repo + SHA-256 checksum	huggingface.co/nova-assistive
9. Register in backend	register_model_in_backend.py	module_id, version, filename, checksum	New row in model_registry table; app discovers update	PostgreSQL backend
10. Mobile OTA update	Flutter SyncService + model registry router	New registry entry	App downloads and hot-swaps model on next launch	Android device